2025 IEEE/ACIS 29th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD) | 979-8-3315-1258-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/SNPD65828.2025.11252585

Semantic Search in a Document Management System Using Retrieval Augmented Generation

Srikant Krishnan
Managing Director
dMACQ Software Pvt. Ltd.
Mumbai, India
srikant@dmacq.com

Nary Subramanian
Department of Computer Science
University of Texas at Tyler
Tyler, USA
nsubramanian@uttyler.edu

***Abstract*— Document Management Systems (DMS) are repositories of documents used in organizations primarily for easy search and retrieval. Native search interfaces of DMS's are usually syntactic that are based on keywords and their logical combinations and when their results are received, the interpretation of the documents in the results is done by the human user. However, with the recent emergence of Large Language Models (LLM) that can generate data and subsequently a more contextual data generation from LLMs using Retrieval Augmented Generation (RAG), we can now perform semantic searches in DMS wherein the queries are in natural language and responses are machine interpreted data from relevant documents in the DMS. In this paper we present our RAG-integrated DMS system, its architecture and its implementation, and show how semantic searches can be more useful and easier to use than the traditional syntactic search mechanism in DMS. For this study we used a commercial DMS from dMACQ and integrated it with a RAG that used ChatGPT as its LLM.**

Keywords—document, management, retrieval, search, syntactic, semantic, augmented, generation

I. INTRODUCTION

Document Management Systems (DMS) [1] store documents and other digital content for easy viewing, retrieval, and search. Their functional requirements are listed in ISO16175-1 [2] and primary requirements include records capture and classification, records retention and disposition, records' integrity and maintenance, and records' discovery and use. As part of records' discovery and use requirement, a DMS should support search, retrieval, presentation, use, and interoperability while including access restrictions and permissions. Moreover, search and retrieval are one of the primary uses of a DMS – once its repositories are populated, its users are usually interested in locating documents of interest to view their contents.

Primary techniques used for document search and retrieval are metadata search of the database or the full content search of documents stored in the repository; such searches typically look for keyword(s) that the user entered in search options and then try to find a match for those keyword(s). Some variants of such searches also support search by other criteria including date ranges or departments or even using stems of keyword(s). Such searches can be classified as syntactic searches in that they are primarily targeting a match based on predefined metadata or content. An example of such a search is finding all documents that came from a specific vendor, say, "ABC Vendor" between January 1st, 2024 through to December 31st, 2024; usually these search choices are entered as keyword(s) in a user interface provided specifically for this purpose in the DMS.

With the development of Large Language Models (LLM) [3] it is possible for artificial intelligence models to process and generate human-like text. It is built using deep learning techniques, particularly transformers, and is trained on massive datasets to understand and generate natural language. Among the most well-known LLM's are ChatGPT [4], Llama [5], and Deepseek [6]. Recently, to make responses from LLM's be more relevant, the technique called Retrieval Augmented Generation (RAG) has been developed [7] – RAG enhances text generation from an LLM by using contextual data for producing a response from the LLM – this contextual data is usually external knowledge retrieved from a document database that is sent to the LLM before the latter generates a response. Instead of relying solely on a language model's internal knowledge, RAG improves accuracy and reduces hallucinations by incorporating pertinent information.

In this paper we discuss our RAG-based search approach where a DMS integrated with a RAG provides the ability for semantic searches as well. Such searches are based on an understanding of the context of the search and searching through relevant documents and/or associated metadata based on the context. For example, we can now enter the search query directly as text "Find all invoices from ABC Vendor between January 1st, 2024 through to December 31st, 2024, that exceed $10,000 in value." Semantic searches allow free form queries to be used, and the RAG will provide only relevant responses – in this case all documents that contain invoices in excess of the specified amount within the given date range will be the response. A further advantage of this semantic search is that it allows repeated queries on previous responses to iteratively optimize search results – so a follow-on query to the above example "Of these invoices which ones contain the part number XYZ" will result in a further condensed list in the output; syntactic searches will need to perform a search for this follow-on query from the beginning since each search query is usually treated independent of each other. Another advantage of this RAG-based search is that role-based access control (RBAC) is possible that can restrict the accesses to this search facility to only those roles that need this feature; moreover, the ability to view documents that match queries can be strictly controlled by RBAC to prevent unintentional disclosure of information to roles not meant to access certain documents. Moreover, a RAG incorporating a feedback system allows the search system to better optimize its responses to the needs of its users.

A demonstration with detailed guidelines on how RAG can be developed is given in [8] where they have used a set of pdf documents to obtain relevant information from ChatGPT and Llama. Our paper integrates RAG to an industrial DMS and we analyze the results from this practical application. The article in [9] discusses how to use an LLM to make conversational queries on the contents of a pdf document. However, our application allows users to make iterated queries over a collection of documents in a DMS that need not be only pdf documents and helps fine-tune the information received in the responses so that more specific details may be obtained. The invention in [10] uses an LLM to get key-value pairs from the input chat which are then used to retrieve appropriate data from the data store. We use LLM for contextual search from within documents in a DMS and a RAG for fine-tuning search using iterated queries. Some of the commercially available DMS's implement a similar functionality with slight differences: for example, Innowise [11], Box [12], and IBM [13] provide chatbots to access documents – difference between these systems and our implementation seem to be primarily in the manner of use and deployment: our implementation allows repeated queries over a set of documents, works on-premise and in the cloud, and allows user-configurable interconnection with an LLM of choice.

However, not many commercial DMS provide an on-premise version that integrates a RAG-based semantic search ability while dMACQ's [14] DMS incorporates this feature. In this paper we discuss the RAG-based search in dMACQ's DMS and explain its architecture, its implementation, and our analysis based on its use.

II. Architecture of DMS Integrated with RAG

A DMS can be either a stand-alone application or a web-based application; if it is a stand-alone application then it works on multiple operating systems and if it is a web-based application then it is accessible from a browser. Either way, a DMS has repositories for storing documents and their metadata. A web-based application can be deployed in a cloud-based environment where scalability and performance can be achieved by using multiple virtual machines. Architecture of a DMS can be seen in [1]. Repositories that a DMS uses to store metadata and documents are usually databases and filesystems.

In this paper we are interested in adding a RAG to a DMS and this architecture is shown in Fig. 1; in this figure, primary components are shown as boxes, and the numbered arrows represent data flows between these components. User interacts with the DMS for uploading documents, syntactic search of documents, viewing documents, or for downloading documents; the user can also perform semantic queries based on the information in the documents stored in the DMS (arrow numbered '1'). For the purpose of integrating with a RAG, the DMS either duplicates its repositories in a vector database or uses a vector database as the primary repository; a vector database permits easier searching of structured and unstructured data using vector embeddings – the contents and the query are embedded in vectors that allows for easier semantic matches; the DMS also retrieves its documents from this database (arrow numbered '2').

To perform semantic queries, the DMS is provided with a chat interface; when the user types a query using any human language into the chat interface, the query is first sent (arrow numbered '3') to the RAG which creates a vector embedding for the query and this embedding is sent to the vector database for retrieving documents relevant to the query (arrow numbered '4'); the RAG then sends the query with the retrieved documents to the Large Language Model (LLM) for response (arrow numbered '5'). The response from the LLM (arrow numbered '6') is then sent to the RAG which then sends the response (arrow numbered '7') to the DMS; the DMS then presents this response to the user ensuring only data appropriate to user's authorizations are displayed. RAG also stores retrieved documents, prior queries, and responses in its cache so that any follow-on queries from the user will have its historical context

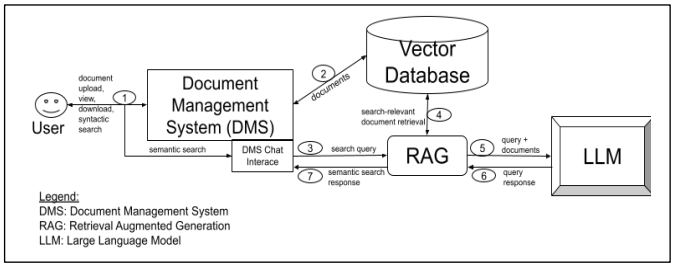

Fig. 1. Document Management System Integrated with Retrieval Augmented System

preserved by sending data in the cache to the LLM for processing; moreover, RAG may implement guardrails to ensure responses from LLM's are appropriate for the user.

III. Application of DMS-RAG for Search and Retrieval

For this study, we integrated the DMS from dMACQ with a RAG; dMACQ's DMS exposed standard API's and the RAG, written in Python, used these API's. The vector database used was Elasticsearch and the LLM used was ChatGPT. The DMS and the RAG ran on Amazon Web Services [15] cloud instance that used 16GB RAM and 8 core CPU.

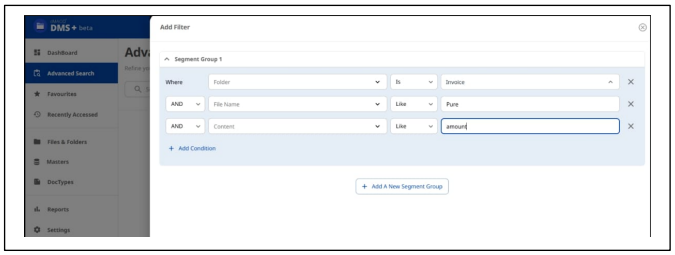

Fig. 2. Keyword-Based Syntactic Search Interface of DMS

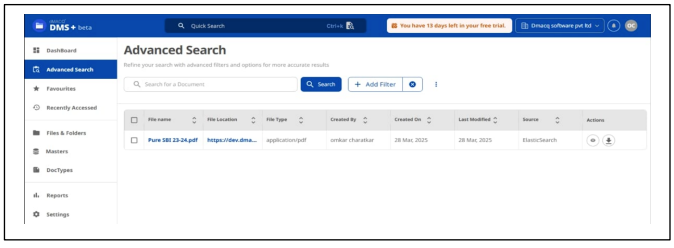

Fig. 3. Search Result for Syntactic Search from DMS

Fig. 2, shows the standard search interface for the DMS where the user enters keywords that can be logically combined

979-8-3315-1258-3/25/$31.00 ©2025 IEEE
SNPD 2025, June 25-27, 2025, Busan, Korea

to fine-tune the search – this is the native search facility provided by the DMS and does not use a RAG. In this search interface, the folder to search for is the 'Invoice', the file name has 'Pure' in it and the content has 'amount' in it. The result of this search is shown in Fig. 3 which is a single pdf file satisfying the search criteria. As can be seen from this syntactic search result, it is up to the user to determine the accuracy of the search and then to interpret the data from the results.

Fig. 4 shows the semantic search query process using a RAG: queries entered by the user are on the right while responses from the RAG are on the left. The user first asks for all files containing "Bank" transactions and the RAG returns a list of files.

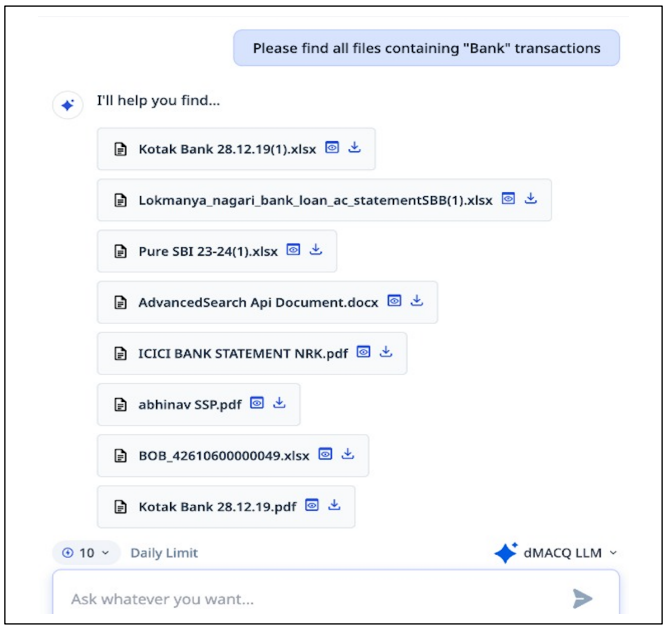

Fig. 4. Semantic Search Query in DMS

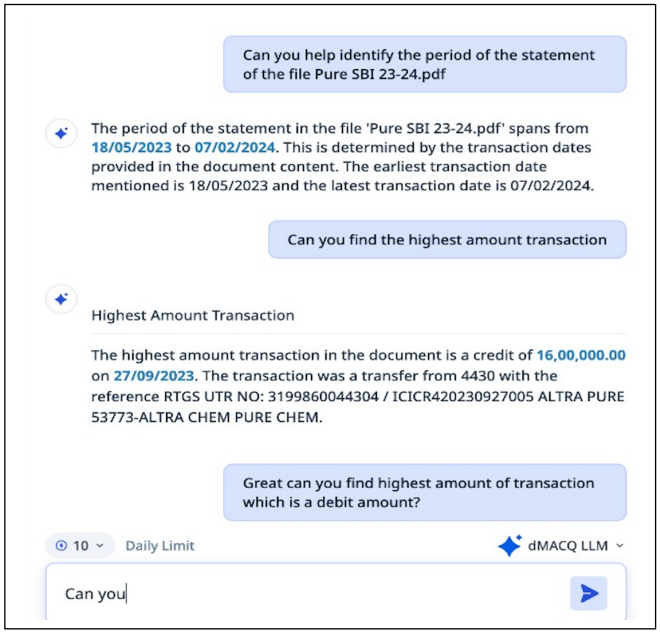

Fig. 5. Secondary Semantic Search Queries in DMS

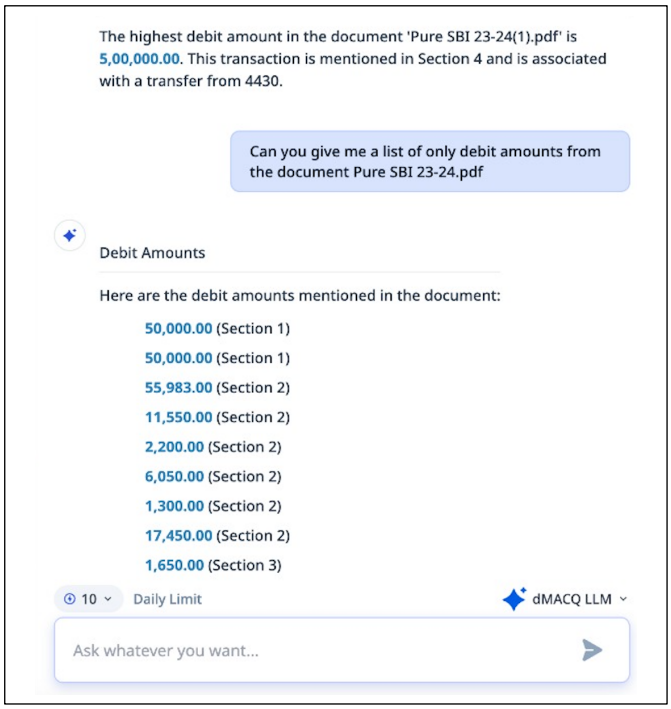

Fig. 6. Further Queries Using Semantic Search in DMS

Fig. 5 shows further queries following the prior search result: so the first query from the user asks to identify the statement period in a specific file from the earlier search result, the file *Pure SBI 23-24.pdf*, to which the RAG reads the contents and returns the period as being from '18/5/2023 to 07/02/2024'. The user then asks to determine the highest amount of transaction in the document which the RAG identifies and provides details. The user then asks to find the highest amount of transaction which is a debit amount, to which RAG's response is shown in Fig. 6. Further, the user then asks to list the debit amounts and the RAG returns responses as shown in Fig. 6. As can be seen, with RAG-based search, the user is able to have a normal chat discussion with the RAG and the results are interpreted by the machine itself.

IV. ANALYSIS OF DMS-RAG IMPLEMENTATION

One important factor that needs to be kept in mind is that appropriate guardrails need to be implemented so that inappropriate queries are not supported; this can be done by informing the LLM beforehand using a system setting that inappropriate content or query should be ignored. Moreover, since DMS is typically used in an enterprise setting, all queries can be logged for any audit of employees' activities.

One of the biggest advantages in semantic search is that data interpretation is done by the RAG on behalf of the user; this is different from keyword-based or syntactic searches where the results must be examined and then interpreted by users. However, the quality of this machine interpretation in a RAG system is based on the quality of data in the vector database; if the documents in the DMS are properly OCR'ed (optical character recognition) and/or scanned at a high resolution or are natively in their electronic forms, then the quality of data in vector database will also be high.

Another advantage of a RAG-based system is that feedback can be provided to the RAG by the user on the quality of RAG's outputs which can influence RAG's responses in the future. This feedback can be incorporated for each response in a series of iterated queries so that RAG's outputs' quality can improve during that interactive session itself.

While our discussion above used ChatGPT for the LLM, there is no restriction on which LLM is used: we tried ChatGPT, Deepseek, and Llama – while ChatGPT and Deepseek provide API's to access them on their own servers, Llama required us to host it on our servers and that can be expensive depending on the speed of processing and the number of GPU's used.

Another advantage of RAG-based system is that the user can ask for summaries of documents as well – this is important if the search results are many and instead of user going over each document, the RAG provides the summaries so that the user can choose to drill into the document(s) of interest. RAG can also extract relevant portions of the document(s) if the user requires this be done.

Furthermore, LLM used in dMACQ's DMS-RAG system is user configurable between ChatGPT, Deepseek, and Llama; this allows flexible deployment of this system either on-premise or on the cloud. As mentioned earlier, for this study we used a RAG using ChatGPT as the LLM of choice and we deployed this system on Amazon Web Services cloud.

Table 1 compares the relative merits of standard syntactic search with the RAG-based semantic search for a DMS. As mentioned in the table, syntactic searches are available by default in a DMS while semantic searches are based on queries entered in a chat interface and not usually available as a standard feature. Search criteria for syntactic searches are keywords while for semantic searches natural language is used; the result of a syntactic search is the list of documents that match the search criteria usually in a tabular form while for the semantic search the list is displayed as a response to the query in the chat interface itself. As discussed earlier, one of the big differences is in the interpretation of results – for syntactic searches interpretation is done by the human user while for semantic searches interpretation is done by the machine itself; this lends itself to the next advantage: ability to fine tune queries based on prior responses in a semantic search which is not typically available in syntactic searches. Using a structured search interface is usually less easy to use than simply typing the search in natural language in a chat interface but semantic queries require appropriate guardrails be installed to avoid inappropriate or unauthorized information from being returned; however, syntactic searches are available by default in a DMS, semantic searches require RAG integration which requires time and effort. However, semantic searches allow additional features to be incorporated such as document summarization and extraction of relevant passages from documents, which are usually not possible in traditional syntactic searches.

Cost is often higher for RAG implementations not only for development but also for deployment since integration with LLM's requires subscriptions for cloud deployments or a high cost of computing resources for locally hosted LLM's; this also impacts performance since semantic searches rely on external system integration often using the Internet while syntactic searches are usually based on local resources that tend to be faster. From a security viewpoint, while for syntactic searches Role-Based Access Control (RBAC) will be sufficient for preventing unauthorized searches or results being displayed, for semantic searches, guardrails will also be required to prevent inappropriate or unauthorized information from being revealed.

TABLE I. COMPARISON OF SYNTACTIC AND SEMANTIC SEARCH FEATURES IN DMS

Feature	**Syntactic Search**	**Semantic Search**
Search Interface	Native search interface that allows logical combination of search criteria	Chat interface
Search Criteria	Keywords	Natural language
Results Display	List of documents in a structured search results user interface	Inside the chat interface itself
Interpretation of Results	By human users	Primarily by machine (RAG)
Ability for Repeated Queries	Limited	Primary advantage – allows asking fine-tuned questions based on prior responses
Ease of Use	Limited	Enhanced since free-form questions can be asked; however, guardrails will need to be in place
Native Support	Yes	Needs RAG integration
Extension of Features	Limited	High – can summarize results or extract relevant information
Cost	Low	High – requires development and maintenance costs
Performance	High	Depends on computing infrastructure or the Internet access speeds
Security	RBAC can control who can search	RBAC and guardrails will be required for privacy and confidentiality

V. CONCLUSION

Document Management Systems (DMS) serve as centralized repositories for organizational documents, facilitating efficient search and retrieval. Traditional DMS search interfaces typically rely on keyword-based queries and logical operators, requiring human users to interpret the retrieved documents manually. However, with the advent of Large Language Models (LLMs) capable of generating and understanding contextual information, and the further

enhancement through Retrieval-Augmented Generation (RAG), semantic search capabilities in DMS have become possible. This allows users to perform queries in natural language while the system generates machine-interpreted responses from the stored documents.

In this paper, we introduce our RAG-enhanced DMS solution, describing its architecture and implementation. We demonstrate how semantic search improves usability and effectiveness compared to conventional keyword-based syntactic search methods. For this study, we utilized a commercial DMS from dMACQ [14] and integrated it with a RAG system powered by ChatGPT as its LLM. One of the biggest advantages we found was that iterated or repeated queries to clarify previous responses from RAG were possible – this allows users to have a normal chat conversation with the RAG to get better fine-tuned responses based on the search.

For the future, we plan to extend the ability of the RAG by providing chat facility in multiple human languages using a neural machine translator to translate user inputs to English and to reverse RAG-responses back to the original language of the user. However, we believe that RAG-based semantic searches will be the default feature in DMS in the future.

REFERENCES

[1] S. Krishnan and N. Subramanian, “Evaluating Carbon-Reducing Impact of Document Management Systems”, IEEE Green Technologies (GreenTech) Conference, New Orleans, LA, April, 2015.

[2] ISO 16175-1:2020 Information and documentation — Processes and functional requirements for software for managing records Part 1: Functional requirements and associated guidance for any applications that manage digital records, Second Edition, July 2020, https://www.iso.org/standard/74294.html.

[3] A. Vaswani et al., “Attention Is All You Need", Version V7, August 2, 2023, arXiv:1706.03762 [Online]. Available: https://arxiv.org/abs/1706.03762.

[4] https://chatgpt.com accessed on March 29, 2025.

[5] https://www.llamaindex.ai/ accessed on March 29, 2025.

[6] https://www.deepseek.com/ accessed on March 29, 2025.

[7] P. Lewis et al., “"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", Version V4, April 12, 2021, arXiv:2005.11401 [Online]. Available: https://arxiv.org/abs/2005.11401.

[8] A. S. Khan, M. T. Hassan, K. K. Kemell, J. Rasku, and P. Abrahamsson, “Developing Retrieval Augmented Generation (RAG) based LLM Systems from PDFs: An Experience Report”, October 21, 2024, arXiv:2410.15944v1 [Online]. Available: https://arxiv.org/html/2410.15944v1#abstract.

[9] R. Dharmavaram, “How To Simplify Document Management With Generative AI”, Forbes, January 16, 2025, https://www.forbes.com/councils/forbestechcouncil/2025/01/16/how-to-simplify-document-management-with-generative-ai/, accessed on March 28, 2025.

[10] **Base64.ai.** "Patent-Pending Document Search Engine." Base64.ai, https://base64.ai/resource/base64-ais-patent-pending-document-search-engine/, accessed on March 28, 2025.

[11] https://innowise.com/case/rag-based-chatbot/ accessed on March 30, 2025.

[12] https://www.box.com/resources/what-is-retrieval-augmented-generation accessed on March 30, 2025.

[13] https://www.ibm.com/think/tutorials/build-multimodal-rag-langchain-with-docling-granite accessed on March 30, 2025.

[14] https://www.dmacq.com accessed on March 28, 2025.

[15] https://aws.amazon.com/ accessed on March 31, 2025.